

昇思+香橙派： 软硬结合玩转DeepSeek蒸馏模型 开发实战

www.huawei.com

前言

2025年蛇年春节，DeepSeek强势出圈，以十分之一的训练成本比肩OpenAI GPT-4o的性能，重塑AI世界新秩序。DeepSeek向全世界开源，为全球的数据科学家、AI爱好者乃至中小开发者开辟了一条通往前沿技术的道路。

由于DeepSeek模型开发本身对算力等有一定的需求，故很多开发者把目光转向了DeepSeek相关的蒸馏模型，此类模型在对资源需求小的前提下，能力依旧出色。

在本次课程中，我们将以DeepSeek蒸馏模型为例，讲解如何基于昇思MindSpore，在香橙派开发板上完成该模型的开发、微调、推理、性能提升，以及分享一些在开发板上实践的经验供大家参考。

目标

- 学完本课程后，您将能够：
 - 了解并熟悉MindSpore和相关套件的基础使用
 - 掌握基于MindSpore的DeepSeek蒸馏模型的开发、微调、推理、性能优化实践全流程
 - 掌握DeepSeek蒸馏模型实践全流程在香橙派开发板上的适配

目录

- 1. 昇思+香橙派+DeepSeek介绍**
- 2. DeepSeek-R1-Distill-Qwen-1.5B 模型开发与适配**
- 3. DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调**
- 4. DeepSeek-R1-Distill-Qwen-1.5B 模型推理和性能优化**

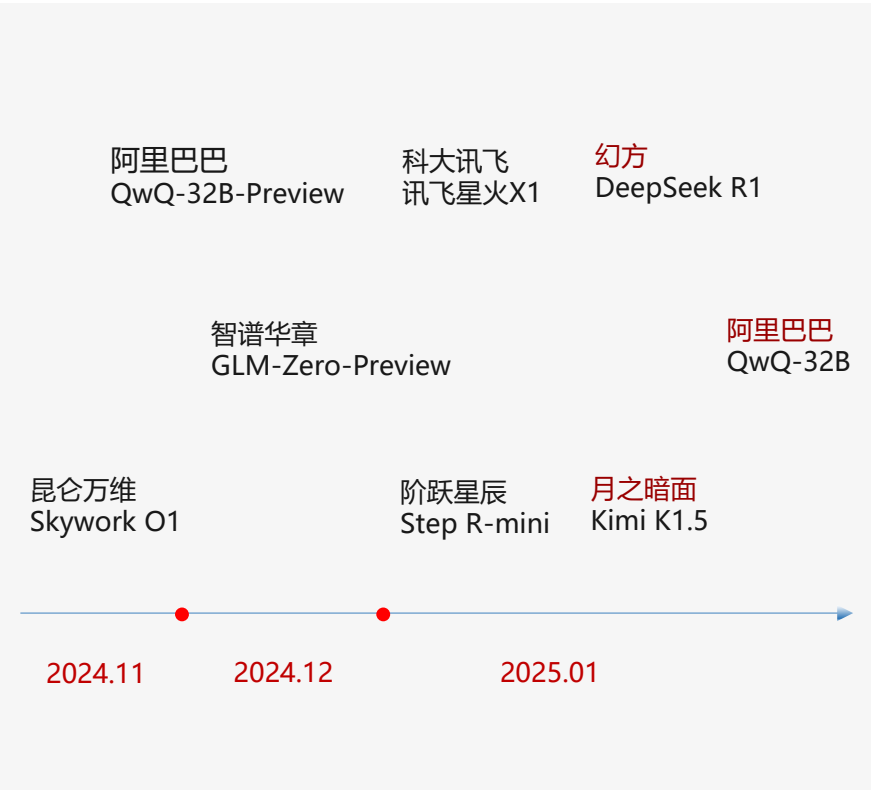
DeepSeek V3/R1轰动全球，带动模型结构和训练方式转变，框架围绕头部模型创新有机会产生1+1>2的效果

变化一：MoE模型结构成为业界主流

变化二：类o1/R1慢思考模型成业界关键探索路径

框架的机会与挑战

作者	模型	发布时间	总参	激活参数	路由专家	共享专家
Mistral	mixtral-8x7B	2024.1	47B	13B	2/8	0
	mixtral-8x22B	2024.4	141B	39B	2/8	0
深度求索	deepseek-v1	2024.1	145B	22B	12/128	4
	deepseek-v2	2024.5	236B	21B	6/160	2
	deepseek-v3	2024.12	671B	37B	8/256	1
阿里巴巴	Qwen2-57a14	2024.6	57B	14B	8/64	8
昆仑万维	skywork-moe	2024.6	146B	22B	2/16	0
微软	phi3.5-moe	2024.8	42B	7B	2/16	0
元象	xverse-moe	2024.9	255B	36B	6/64	2
腾讯	Hunyuan-large	2024.11	389B	52B	1/16	1
MiniMax	MiniMax-Text-1	2025.1	456B	46B	2/32	0



- 预训练场景**

自研模型持续演进，吸收DS先进开源技术

挑战

 - 细粒度专家模式下专家负载不均、不可掩盖通信占比40%+
 - 昇腾亲和的DS MoE相关优化技术
- 后训练场景**

复现R1训练流程并将流程与强化学习算法应用于自研慢思考模型

挑战

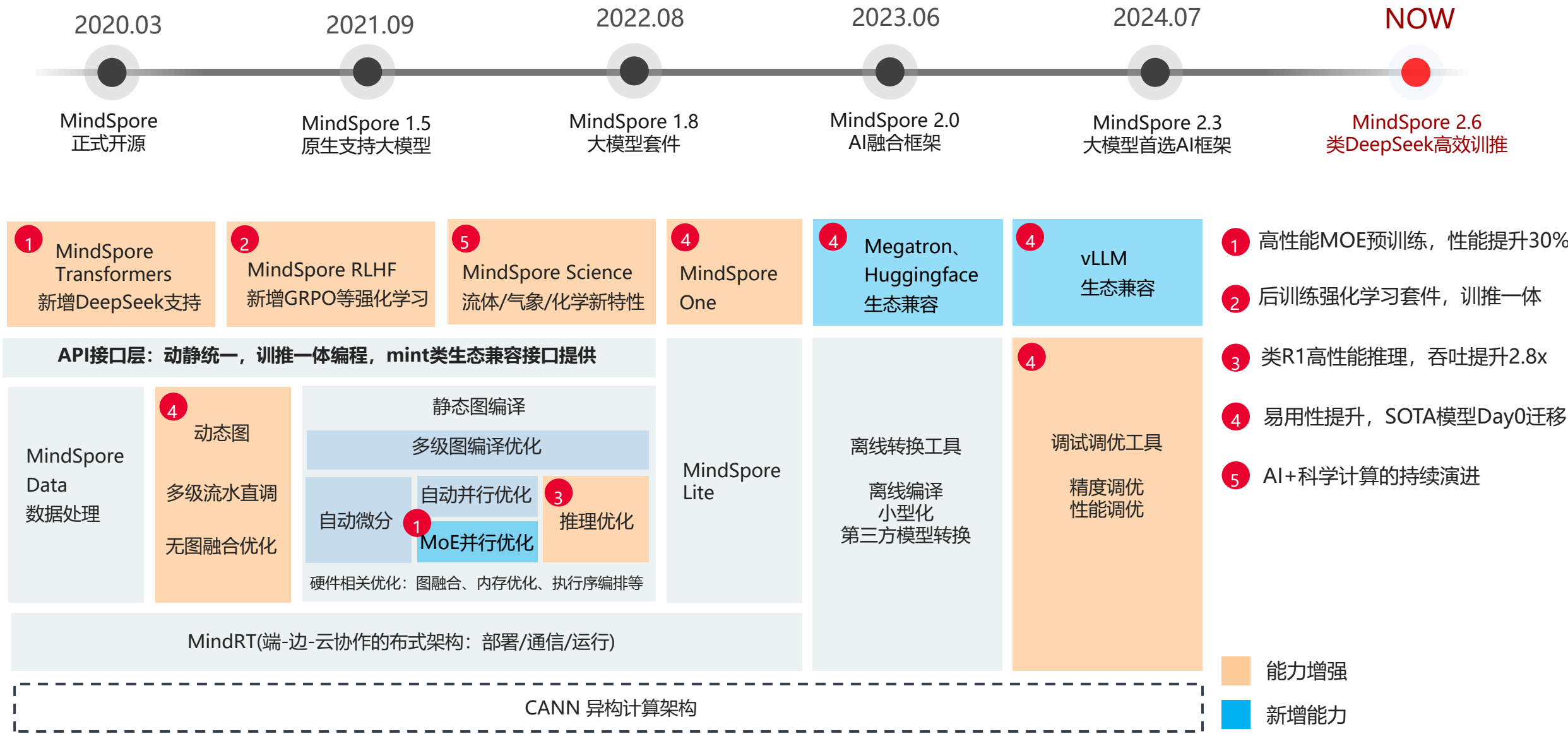
 - 训推倒换时间小时级、推理算力开销占比高达60%+
- 推理部署场景**

MAAS、行业智能化、数据蒸馏...

挑战

 - 千亿MoE模型推理PD实例间、专家间负载不均，All2All通信占比高
 - 时延、吞吐、精度、成本等指标的提升

昇思MindSpore 2.6全面支持类DeepSeek V3/R1 MoE模型高性能训推



昇思 + DeepSeek实践进展

训推全面支持、开箱即用！DeepSeek V3昇思MindSpore版本上线开源社区

近日，基于昇腾AI硬件与昇思MindSpore AI框架版本的DeepSeek-V3完成开发支持并上线昇思开源社区，面向开发者提供开箱即用的预训练和推理能力，并已成功在大规模集群上预训练和部署。

上海交通大学联合MindSpore与openEuler社区，实现DeepSeek全栈开源单机推理部署

携手北京大学，MindSpore+openEuler打造支持vLLM的DeepSeek全栈开源推理方案

昇思MindSpore 2025年03月09日 21:00 广东

从“算力自主”到“普惠共赢”，中国算力网为DeepSeek云服务提供坚实的底座

OpenI启智社区 2025年02月08日 18:30 广东

值得一提的是，在鹏城实验室与华为昇思MindSpore及华为昇腾团队的通力协作下，我们已完成DeepSeek部分模型在昇腾NPU上的适配工作，这些模型现已能

[M] 昇思 MindSpore 安装 学习 文档 产品全景 资源 社区 活动与资讯 昇思大模型平台 HOT 搜索文档

新闻 > 人人都能上手部署DeepSeek模型：Janus-Pro香橙派开发板实践指南

人人都能上手部署DeepSeek模型：Janus-Pro香橙派开发板实践指南

昇思框架×昇腾开发板：DeepSeek蒸馏版与多模态模型部署全流程实践

3月7日，在昇腾AI开发者创享日·浙江站分论坛上，以“用昇思框架玩转DeepSeek开发实战”为主题的昇腾开发板集训营火热开课。活动吸引了来自高校师生与企业技术骨干的550余名开发者踊跃报名，最终70余名开发者通过筛选获得现场参与资格，共同开启昇腾开发板深度实践之旅。

人人都能上手部署DeepSeek-R1蒸馏模型：社区开发者应用昇思+香橙派 AI Pro实践优化心得

作者：陈新杰 郑州轻工业大学 梅科尔工作室

昇腾开发板 - OrangePi Alpro



- 香橙派OrangePi Alpro采用昇腾AI技术路线，具体为4核64位处理器+AI处理器，集成图形处理器，支持8/20 TOPS AI算力，广泛适用于AI边缘计算、深度视觉学习及视频流AI分析等各个行业。满足大多数AI算法原型验证、推理应用开发的需求。

香橙派开发板相关资料的获取方式

官方资料



官方镜像



样例丰富，易学易用

香橙派论坛(<http://forum.orangepi.cn/>)、昇腾社区 (hiascend.com)、昇思社区 (mindspore.cn) 为开发者免费提供数百个代码参考样例。



香橙派官方资料



昇腾开发板专区



昇思开发板开发教程

昇思MindSpore+ 香橙派技术能力进展

- **开发友好：**动态图易用性提升，类huggingface风格降低开发调试门槛
- **性能提升：**mindspore.jit编译成图，三行代码实现推理性能提升一倍
- **全流程支持：**在香橙派上支持模型训推

```
prompts = [  
    "Simply put, the theory of relativity states that ",  
    "My favorite all time favorite condiment is ketchup.",  
]
```

```
NUM_TOKENS_TO_GENERATE = 40
```

```
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"  
tokenizer = LlamaTokenizer.from_pretrained(model_id)  
model = LlamaForCausalLM.from_pretrained(model_id, ms_dtype=mindspore.float16)
```

```
# quantize_cfg = w8x8(model.model.config)  
# quantize(model, cfg=quantize_cfg)
```

```
model.jit()  
  
inputs = tokenizer(prompts, return_tensors="ms", padding=True)  
set_pyboost(False)  
  
@mindspore.jit(jit_config=mindspore.JitConfig(jit_syntax_level='STRICT'))  
def decode_one_tokens(model, cur_token, input_pos, cache_position, past_key_values):  
    logits = model(  
        cur_token,  
        position_ids=input_pos,  
        cache_position=cache_position,  
        past_key_values=past_key_values,  
        return_dict=False,  
        use_cache=True  
    )[0]  
    new_token = ops.argmax(logits[:, -1], dim=-1)[:, None]  
    return new_token
```

- DeepSeek-R1-Distill-Qwen-1.5B
- DeepSeek-Janus-Pro-1B
- Qwen1.5-0.5b
- TinyLlama-1.1B
- MiniCPM3-4B
- DctNet
- ResNet50
- ViT
- FCN
- ShuffleNet
- SSD
- RNN
- LSTM+CRF
- GAN
- DCGAN
- Pix2Pix
- Diffusion
- ResNet50_transfer



orange-pi-mindspore案例代码仓

项目介绍

项目简介：基于《甄嬛传》中提取出的甄嬛与他人对话的数据集，对DeepSeek-R1-Distill-Qwen-1.5B模型进行LoRA微调，使得微调后模型可以模仿甄嬛说话。

项目核心目标：基于MindSpore NLP套件，快速构建DeepSeek-R1-Distill-Qwen-1.5B模型的微调+推理全流程工程，为开发者提供香橙派开发板上大模型工程实践的可复用参考。

核心解决问题：针对香橙派昇腾硬件的算子缺失问题（含CumSum、Gather等核心计算关联算子），提供框架层解决方案；同时覆盖开发板端内存占用优化、推理性能提升等关键实践要点。

课程核心模块：

1. 模型开发与适配：前置定位模型缺失算子，提供框架层解决方案，保障模型可运行
2. 模型微调：快速搭建LoRA微调工程，针对性优化内存/显存
3. 模型推理：构建高效推理工程，对比微调前后模型效果
4. 推理性能优化：探索并实践开发板端推理性能提升方案

目录

1. 昇思+香橙派+DeepSeek介绍
2. DeepSeek-R1-Distill-Qwen-1.5B 模型开发与适配
3. DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调
4. DeepSeek-R1-Distill-Qwen-1.5B 模型推理和性能优化

DeepSeek-R1-Distill-Qwen-1.5B介绍

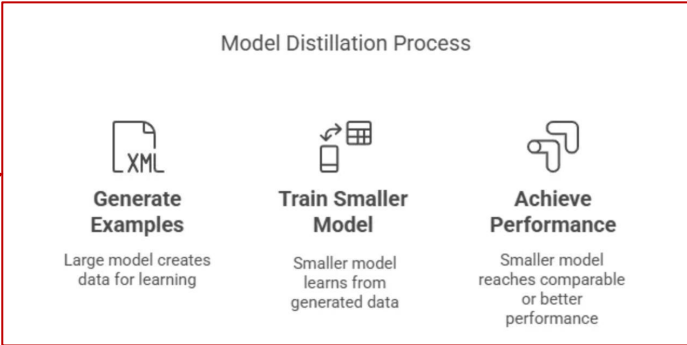
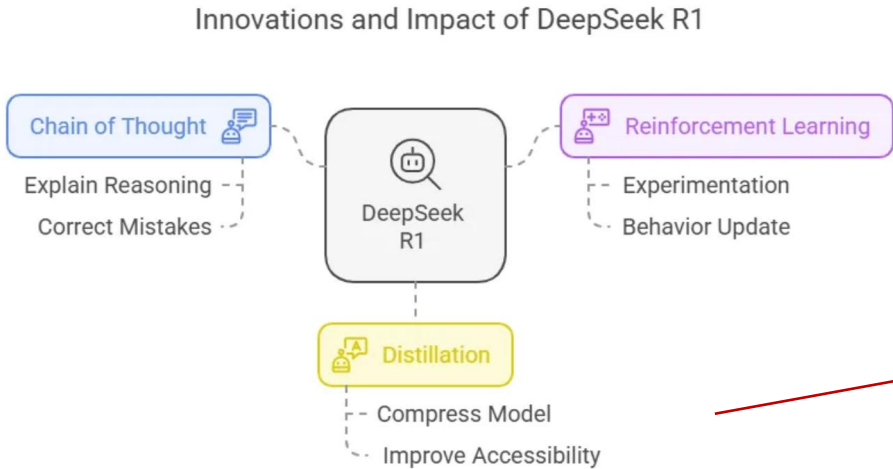
虽然 V3 和 R1 实力强大，但由于需要 8 张 141GB 显存的 NVIDIA H200 GPU，运行成本高，不适合所有人。

为此，DeepSeek 推出了蒸馏版模型，将 R1 的推理能力以更轻量方式呈现。它们不是从头训练，而是：

- 选用 Llama 3.1/3.3 和 Qwen 2.5 的 6 个开源模型
- 用 R1 生成 80 万条高质量推理数据
- 基于这些数据进行监督微调（SFT，无 RL 阶段）

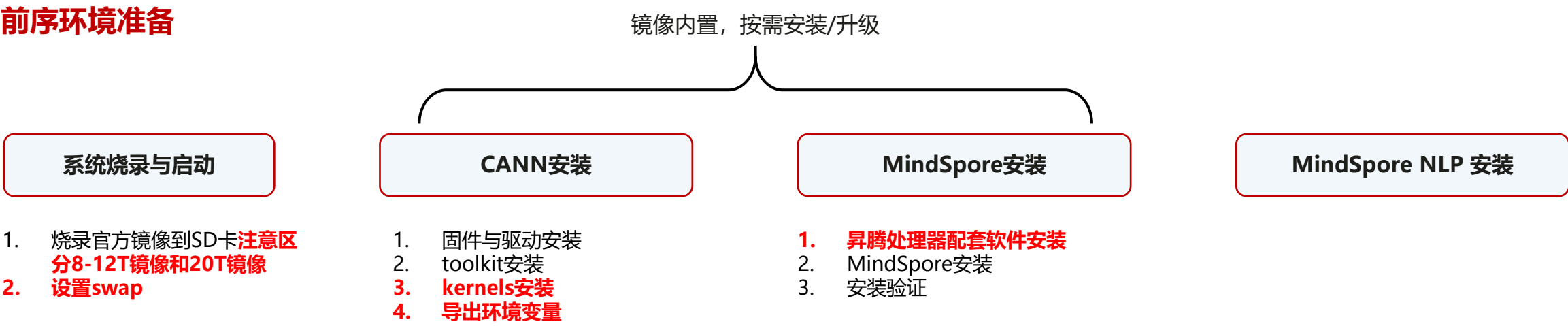
这些模型更易部署，推理能力也依然出色。而DeepSeek-R1-Distill-Qwen-1.5B是最小规模DeepSeek蒸馏模型。

Model	Base model	Best for	Reasoning strength	Compute cost
DeepSeek-R1-Distill-Qwen-1.5B	Qwen2.5-Math-1.5B	Entry-level reasoning, basic math	👍👍	Low
DeepSeek-R1-Distill-Qwen-7B	Qwen2.5-Math-7B	Mid-tier math & logic tasks	👍👍👍	Medium
DeepSeek-R1-Distill-Llama-8B	Llama-3.1-8B	Mid-tier math & logic tasks, coding assistance	👍👍👍	Medium
DeepSeek-R1-Distill-Qwen-14B	Qwen2.5-14B	Advanced math & logic tasks, problem-solving, coding assistance	👍👍👍👍	Medium-High
DeepSeek-R1-Distill-Qwen-32B	Qwen2.5-32B	Complex math, logic, & coding tasks, problem-solving, research	👍👍👍👍👍	High
DeepSeek-R1-Distill-Llama-70B	Llama-3.3-70B-Instruct	Complex math, logic, & coding tasks, problem-solving, research	👍👍👍👍👍👍	High



DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

前序环境准备



环境准备CheckList

香橙派开发板	OrangePi Alpro 20T 24G
CANN	社区版 8.0.0beta1 （镜像内置）
MindSpore	2.5.0 （镜像内置2.4.10，需手动升级）
MindSpore NLP	0.4分支源码编译安装： pip install git+https://github.com/mindspore-lab/mindnlp.git@0.4 网络不好时可使用如下命令： pip install git+https://gitee.com/mindspore-lab/mindnlp.git@0.4

环境准备详细指南可参考
昇思官网-教程-香橙派开发-环境搭建指南：



DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

MindSpore NLP结构简介

MindSpore NLP是面向NLP领域的昇思MindSpore套件，旨在做到面向Hugging Face生态的无缝兼容，为开发者提供丰富的Hugging Face模型生态的同时，兼顾昇思MindSpore框架的强大加速能力。

在正式开始算子缺失问题的定位之前，我们先来简单了解下MindSpore NLP（0.4分支）的结构。

```
mindnlp (0.4分支, 代码仓根目录)
├── (其他目录: examples/、llm/、docs/等)
├── mindnlp/
│   ├── core/
│   ├── transformers/
│   └── ... (其他子目录)
└── tests/
    ├── transformers/
    └── ... (其他子目录)
```

核心实现模块:

- 对标Hugging Face实现了transformers、peft、accelerate等开发库
- 对框架接口进行封装，面向开发者提供统一接口
- ...

在本课程中我们重点关注**transformers**和**core**这两个目录。

测试模块:

对标HuggingFace，集成了针对不同库的单元测试
在本课程中我们重点关注**transformers**这个目录。

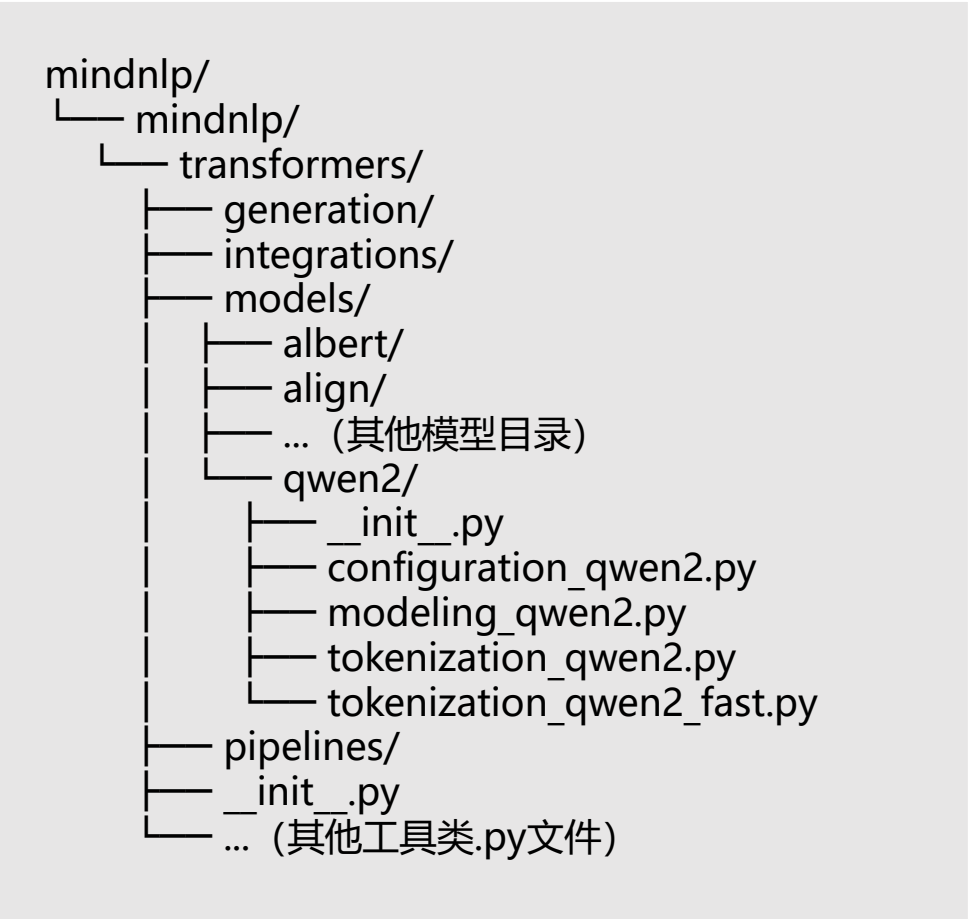


更多详细信息请见MindSpore NLP代码仓（0.4分支）

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

MindSpore NLP结构简介

mindnlp/mindnlp/transformers



MindSpore NLP 的mindnlp/mindnlp/transformers目录结构与 Hugging Face Transformers的src/transformers模块高度相似：模型实现均集成在models路径下，按模型名划分独立目录，每个模型目录核心包含三类文件：

configuration_xxx.py	模型配置文件，定义并管理所有可配置参数
modeling_xxx.py	模型结构与推理文件，实现网络结构、前向传播及各类预训练任务头， 算子缺失的相关适配改动主要在此完成
tokenization_xxx.py	文本预处理核心文件，负责文本与数字 token 的双向转换

(注：xxx 代表具体模型名称)

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

MindSpore NLP结构简介

mindnlp/tests

MindSpore NLP 的mindnlp/tests/transformers目录结构与mindnlp/mindnlp/transformers一致：模型测试文件按模型划分至models目录，test_modeling_xxx.py验证模型推理、训练等基础功能。

test_modeling_qwen2.py为例，核心测试模块为Qwen2ModelTest和Qwen2IntegrationTest。

```
mindnlp/tests/  
├── README.md  
├── __init__.py  
├── ... (其他测试工程)  
└── transformers/  
    ├── ... (其他transformers测试工程)  
    └── models/  
        ├── __init__.py  
        ├── qwen2/  
        │   ├── __init__.py  
        │   └── test_modeling_qwen2.py  
        ├── albert/  
        ├── align/  
        └── ... (其他模型测试目录)
```

```
class Qwen2ModelTest  
    const all_model_classes  
    const all_generative_model_classes  
    const pipeline_model_mapping  
    const test_headmasking  
    const test_pruning  
    const fx_compatible  
    func is_pipeline_test_to_skip  
    func setUp  
    func test_config  
    func test_model  
    func test_model_various_embed...  
    func test_Qwen2_sequence_clas...  
    func test_Qwen2_sequence_clas...  
    func test_Qwen2_sequence_clas...  
    func test_Qwen2_token_classific...  
    func test_save_load_fast_init_fro...  
    func test_past_key_values_format
```

```
class Qwen2IntegrationTest  
    func test_model_450m_logits  
    func test_model_450m_generati...  
    func test_model_450m_long_pro...  
    func test_speculative_generation
```

Qwen2IntegrationTest：集成测试，加载预训练权重验证实际场景下的功能正确性，通过配置RUN_SLOW环境变量启动

Qwen2ModelTest：模型基础功能测试

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

MindSpore NLP套件中有针对模型的ut测试，可通过pytest来测试模型在香橙派的训练和推理，并进行问题定位和调试

- a. 由于MindSpore动态图下框架存在多线程异步行为，所以会出现python调用栈不准确的场景，为了精准定位在test_modeling_qwen2.py脚本中import mindspore之后的位置，加入如下代码，重新跑pytest，查看具体的报错位置并根据报错信息修改。

```
mindspore.set_context(pynative_synchronize=True)
```

- b. 设置环境变量：

```
export RUN_SLOW=True
```

- c. 执行命令：

```
pytest -v -s tests/transformers/models/qwen2/test_modeling_qwen2.py
```

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

开启同步前后的差异对比

测试用例

```
pytest -s -v tests/transformers/models/qwen2/test_modeling_qwen2.py::Qwen2ModelTest::test_new_cache_format_0
```

未开启同步

```
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/transformers/models/qwen2/modeling_qwen2.py", line 175, in rotate_half
    x1, x2 = ops.split(x, x.shape[-1] // 2, dim=-1)
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/core/ops/array.py", line 218, in split
    return mindspore.mint.split(tensor, split_size_or_sections, dim)
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindspore/mint/_init_.py", line 1827, in split
    return ops.function.array_func.split_ext(tensor, split_size_or_sections, dim)
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindspore/ops/function/array_func.py", line 5318, in split_ext
    res = split_tensor(tensor, split_size, dim)
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindspore/ops/auto_generate/gen_ops_prim.py", line 18538, in __call__
    return _convert_stub(pyboost_split_tensor(self, [input, split_size, dim]))
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindspore/common/_stub_tensor.py", line 223, in _convert_stub
    elements = stub.get_elements()
RuntimeError: aclnnCumsumGetWorkspaceSize call failed, please check!
```

可以初步判断出来是ACLNN Cumsum算子的问题，但追溯上文信息并未看到cumsum接口出现，无法判断具体哪一行代码报错

VS

开启同步

```
Exception in thread Thread-7:
Traceback (most recent call last):
  File "/usr/local/miniconda3/lib/python3.9/threading.py", line 954, in _bootstrap_inner
    self.run()
  File "/usr/local/miniconda3/lib/python3.9/threading.py", line 892, in run
    self._target(*self._args, **self._kwargs)
  File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/core/utils/_contextlib.py", line 117, in decorate_context
    return func(*args, **kwargs)
  File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/transformers/generation/utils.py", line 2031, in generate
    result = self._sample(
  File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/transformers/generation/utils.py", line 3036, in _sample
    model_inputs = self.prepare_inputs_for_generation(input_ids, **model_kwargs)
  File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/transformers/models/qwen2/modeling_qwen2.py", line 803, in prepare_inputs_for_generation
    position_ids = attention_mask.int().cumsum(-1) - 1
RuntimeError: aclnnCumsumGetWorkspaceSize call failed, please check!
```

可以明确定位到，是modeling_qwen2.py中803行计算位置编码处用到了cumsum

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

问题分析

测试用例

```
pytest -s -v tests/transformers/models/qwen2/test_modeling_qwen2.py::Qwen2ModelTest::test_new_cache_format_0
```

继续看下方最终的报错信息，发现是当前CANN版本下该昇腾硬件目前还未支持ACLNN Cumsum算子，也就是我们之前说的算子缺失问题。

```
RuntimeError: aclnnCumsumGetWorkspaceSize call failed, please check!

-----
- Ascend Error Message:
-----
EZ1001: [PID: 3637] 2025-03-20-15:53:30.036.146 support for Ascend310B is not implemented[THREAD:4476]

(Please search "CANN Common Error Analysis" at https://www.mindspore.cn for error code description)

-----
- C++ Call Stack: (For framework developers)
-----
mindspore/ops/kernel/ascend/pyboost/customize/cumsum_ext.cc:47 operator()
```

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

问题分析

测试用例

```
pytest -s -v tests/transformers/models/qwen2/test_modeling_qwen2.py::Qwen2ModelTest::test_new_cache_format_0
```

出现如下格式的报错信息，通常都是当前CANN版本下香橙派开发板昇腾硬件不支持XXX算子。

```
# 报错信息一，此处XXX指代算子名称
RuntimeError: aclnnXXXGetWorkspaceSize call failed,
please check!
# 报错信息二，此处XXX指代算子名称
Op XXX does not has any binary.
# 报错信息三，此处XXX指代算子名称
support for Ascend310B is not implemented.
```

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

问题分析

CANN 层负责开发 ACLNN/ACLOP 算子，MindSpore 框架层通过 mint/ops 接口调用对应算子

```
File "/home/HwHiAiUser/.local/lib/python3.9/site-packages/mindnlp/train/wen2.py", line 803, in prepare_inputs_for_generation
    position_ids = attention_mask.int().cumsum(-1) - 1
RuntimeError: aclnnCumsumGetWorkspaceSize call failed, please check!
```

报错定位到Tensor.cumsum()操作，该方法与 mint 接口同属高阶体系，底层默认调用未被香橙派昇腾硬件支持的 aclnnCumsum 算子。

解决方案

需切换到兼容性更高的 ACLOP 算子库 —— 对应框架层即更换到ops接口。



DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

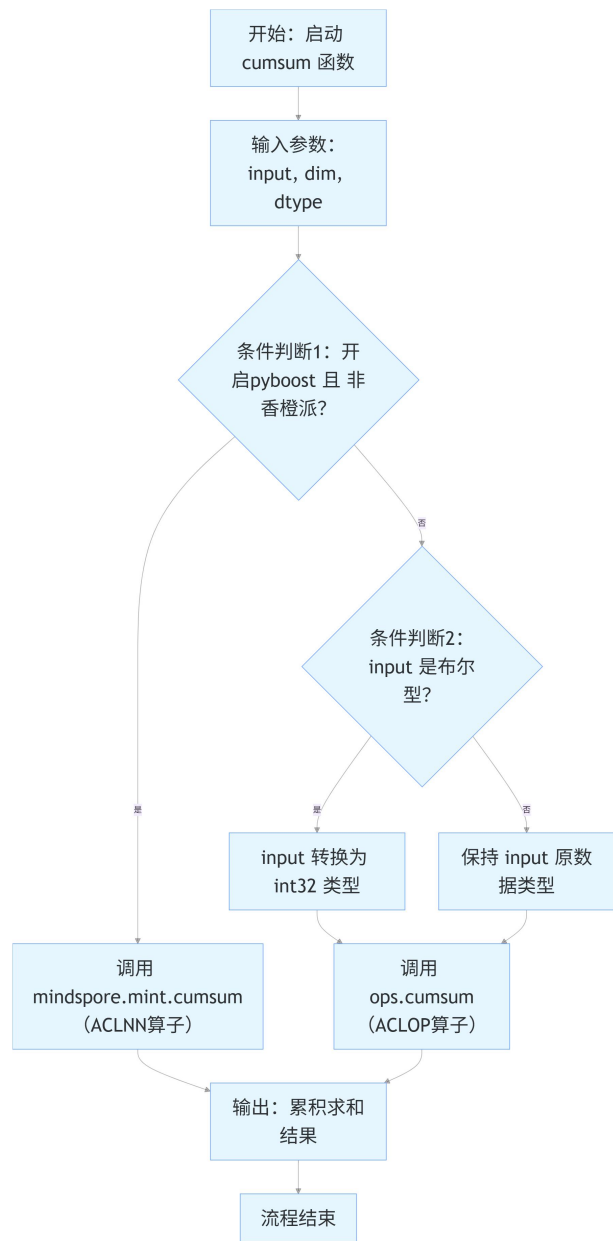
问题修复

MindSpore NLP的mindnlp/core模块下，已完成了对mint和ops接口的同一对外呈现，并可根据硬件条件（ON_ORANGE_PI）判断是否切换为ops接口。

以cumsum实现为例：

```
# cumsum
has_cumsum = hasattr(mindspore.mint, 'cumsum')
def cumsum(input, dim, dtype=None):
    if use_pyboost() and not ON_ORANGE_PI: # since cann8.0 community remove aclnn cumsum
        return mindspore.mint.cumsum(input, dim, dtype)
    if input.dtype == mindspore.bool_:
        input = input.to(mindspore.int32)
    return ops.cumsum(input, dim, dtype)
```

mindnlp/core/ops/other.py中对cumsum的实现



DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

网络调试

问题修复

因此在modeling_qwen2.py中，将对应的Tensor.cumsum修改为调用mindnlp.core中的ops.cumsum即可。

932	940		position_ids = kwargs.get("position_ids", None)
933	941		if attention_mask is not None and position_ids is None:
934	942		# create position_ids on the fly for batch generation
935	-		position_ids = attention_mask.int().cumsum(-1) - 1
	943	+	# position_ids = attention_mask.int().cumsum(-1) - 1
	944	+	position_ids = ops.cumsum(attention_mask.int(), -1) - 1
936	945		position_ids.masked_fill(attention_mask == 0, 1)
937	946		if past_key_values:
938	947		position_ids = position_ids[:, -input_ids.shape[1] :]

为节约时间，验证时可单点执行对应ut，确认问题是否修复，不必运行整个Test_modeling_qwen2.py文件。

```
pytest -s -v tests/transformers/models/qwen2/test_modeling_qwen2.py::Qwen2ModelTest::test_new_cache_format_0
```

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

其他常见问题与解决思路

a. 针对损失函数报错的处理方式

进一步查阅文档，会发现如下注意点：

1. 该损失函数的labels输入需要one-hot格式
2. logits和labels的数据类型需要一致
3. 输出并不是一个零维Tensor
4. 的loss值，而是一个(loss, dlogits)的tuple，且loss是一个Tensor

CLASS mindspore.ops.SoftmaxCrossEntropyWithLogits [\[源代码\]](#)

1

使用one-hot编码获取预测值和真实之间的softmax交叉熵。

SoftmaxCrossEntropyWithLogits算法的更新公式如下：

$$p_{ij} = \text{softmax}(X_{ij}) = \frac{\exp(x_i)}{\sum_{j=0}^{N-1} \exp(x_j)}$$
$$\text{loss}_{ij} = - \sum_j Y_{ij} * \ln(p_{ij})$$

其中 X 代表 logits。 Y 代表 label。 loss 代表 output。

输入：

- **logits** (Tensor) - 输入预测值，其shape为 (N, C) ，数据类型为float16或float32。
- **labels** (Tensor) - 输入真实值，其shape为 (N, C) ，数据类型与 logits 的相同。

输出：

3

两个Tensor(loss, dlogits)组成的tuple， loss 的shape为 $(N,)$ ， dlogits 的shape与 logits 的相同。

异常：

- **TypeError** - logits 或 labels 的数据类型既不是float16也不是float32。
- **TypeError** - logits 或 labels 不是Tensor。
- **ValueError** - logits 的shape与 labels 的不同。

支持平台：

Ascend GPU CPU

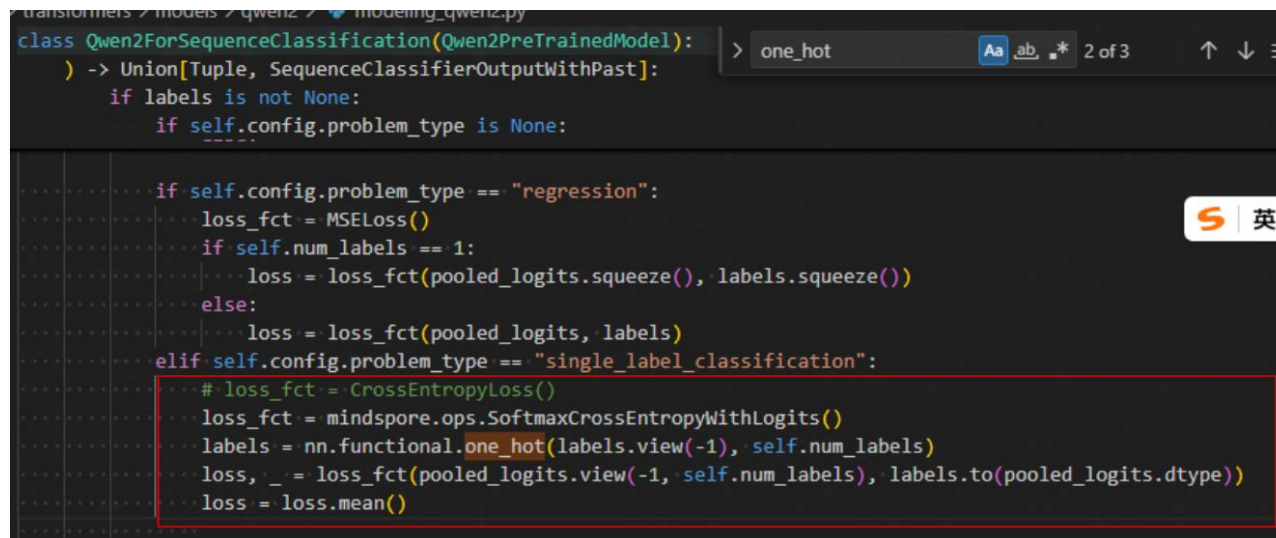
DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

其他常见问题与解决思路

b. 针对损失函数报错的处理方式

针对上述三点，需要对代码进行如下修改：

1. 将损失函数(loss_fct)修改为
mindspore.ops.SoftmaxCrossEntropyWithLogits
2. labels修改为one-hot格式
3. 统一labels和logits的数据类型，计算获取loss
4. 按照原本默认的reduction=' mean' 转成零维Tensor



```
transformers / models / qwen2 / modeling_qwen2.py
class Qwen2ForSequenceClassification(Qwen2PreTrainedModel):
    ) -> Union[Tuple, SequenceClassifierOutputWithPast]:
        if labels is not None:
            if self.config.problem_type is None:
                ...
            elif self.config.problem_type == "regression":
                loss_fct = MSELoss()
                if self.num_labels == 1:
                    loss = loss_fct(pooled_logits.squeeze(), labels.squeeze())
                else:
                    loss = loss_fct(pooled_logits, labels)
            elif self.config.problem_type == "single_label_classification":
                # loss_fct = CrossEntropyLoss()
                loss_fct = mindspore.ops.SoftmaxCrossEntropyWithLogits()
                labels = nn.functional.one_hot(labels.view(-1), self.num_labels)
                loss, _ = loss_fct(pooled_logits.view(-1, self.num_labels), labels.to(pooled_logits.dtype))
                loss = loss.mean()
```

DeepSeek-R1-Distill-Qwen-1.5B在香橙派上的开发与适配

经验分享

c. 针对香橙派上Tensor索引/切片报错的处理方式

执行测试用例

pytest -s -v tests/transformers/models/qwen2/test_modeling_qwen2.py::Qwen2ModelTest::test_beam_search_generate_dict_outputs_use_cache

```
model = model_class(config).eval()
beam_kwargs = self._get_beam_kwargs()

config.is_decoder = True
model = model_class(config).eval()
output_generate = self._beam_search_generate(
    model=model,
    input_ids=input_ids,
    attention_mask=attention_mask,
    inputs_dict=inputs_dict,
    beam_kwargs=beam_kwargs,
    output_scores=True,
    output_logits=True,
    output_hidden_states=True,
    output_attentions=self.has_attentions,
    return_dict_in_generate=True,
    use_cache=True,
)

.../generation/test_utils.py:678:
.../generation/test_utils.py:312: in _beam_search_generate
    output_generate = model.generate(
.../mindnlp/core/utils/_contextlib.py:117: in decorate_context
    return func(*args, **kwargs)
.../mindnlp/transformers/generation/utils.py:2069: in generate
    result = self._beam_search(
.../mindnlp/transformers/generation/utils.py:3462: in _beam_search
    sequence_outputs = beam_scorer.finalize(
.../mindnlp/transformers/generation/beam_search.py:332: in finalize
    indices[i, : len(best_idx)] = best_idx
/home/HuHuiUser/.local/lib/python3.9/site-packages/mindspore/common/tensor.py:527: in _getitem
    return tensor_operator_registry.get(_tensor_setitem)(self, index, value)

self = Tensor(shape=[2, 10], dtype=Int64, value=
[[[-1, -1, -1 ... -1, -1, -1],
  [-1, -1, -1 ... -1, -1, -1]]], index = (0, slice(None, 3, None))
value = (Tensor(shape=[], dtype=Int64, value= 0), Tensor(shape=[], dtype=Int64, value= 0), Tensor(shape=[], dtype=Int64, value= 0))

def _tensor_setitem(self, index, value):
    """Handle tensor setitem"""
    print("self=====", self)
    print("value=====", value)
    if not isinstance(value, Tensor):
        if isinstance(value, (bool, int, float)):
            value = Tensor(value, dtype=self.dtype)
        else:
            raise TypeError(f"Can't assign a {type(value)} to a {self.dtype}.")
    TypeError: Can't assign a <class 'tuple'> to a Int64.

/home/HuHuiUser/.local/lib/python3.9/site-packages/mindspore/ops/composite/multitype_ops/_compile_utils.py:405: TypeError
```

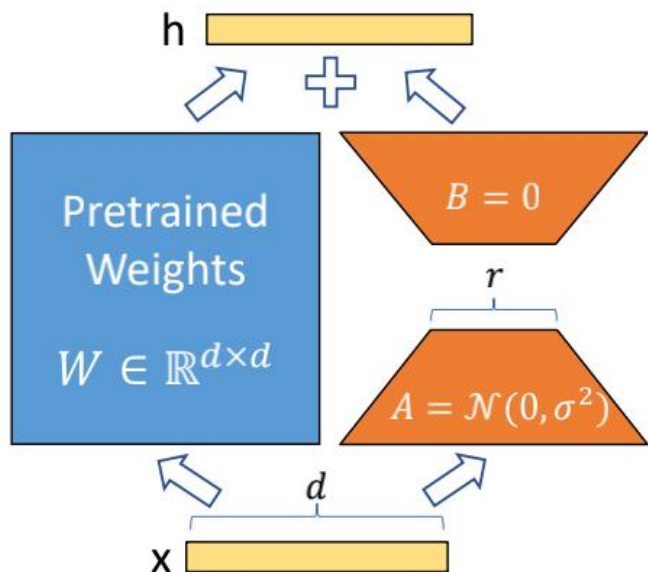
```
mindnlp > transformers > generation > beam_search.py > BeamSearchScorer > finalize
57 class BeamSearchScorer(BeamScorer):
248     def finalize(
328         ...: raise ValueError("'pad_token_id' has to be defined")
329         ...: decoded = decoded.fill(pad_token_id)
330
331         ...: if indices is not None:
332             ...: indices = indices.fill(-1)
333
334         ...: # fill with hypotheses and eos_token_id if the latter fits in
335         ...: for i, (hypo, best_idx) in enumerate(zip(best, best_indices)):
336             ...: decoded[i, : sent_lengths[i]] = hypo
337
338         ...: if indices is not None:
339             ...: # indices[i, : len(best_idx)] = best_idx
340             ...: indices[i, : len(best_idx)] = mindspore.Tensor(best_idx, dtype=indices.dtype)
341
342         ...: if sent_lengths[i] < sent_max_len:
343             ...: # inserting only the first eos_token_id
344             ...: decoded[i, sent_lengths[i]] = eos_token_id[0]
345
346         ...: return UserDict(
```

原代码中的best_idx的类型为tuple，需要转换成Tensor

目录

1. 昇思+香橙派+DeepSeek介绍
2. DeepSeek-R1-Distill-Qwen-1.5B 模型开发与适配
3. DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调
4. DeepSeek-R1-Distill-Qwen-1.5B 模型推理和性能优化

LoRA微调介绍



1

```
model_id = "TJHsiao/DeepSeek-R1-Distill-Qwen-1.5B-FP16"

base_model = AutoModelForCausalLM.from_pretrained(model_id, mirror="modelers", ms_dtype=mindspore.float16)
base_model.generation_config = GenerationConfig.from_pretrained(model_id, mirror="modelers")

base_model.generation_config.pad_token_id = base_model.generation_config.eos_token_id
```

2

```
config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    inference_mode=False, # 训练模式
    r=8, # Lora 秩
    lora_alpha=32, # Lora alaph, 具体作用参见 Lora 原理
    lora_dropout=0.1# Dropout 比例
)
```

3

```
# 实例化LoRA模型
model = get_peft_model(base_model, config)
# 获取模型参与训练的参数, 发现仅占总参数数量的0.5%
model.trainable_params()
```

冻结原始网络参数，对Attention层中QKV等添加旁支，包含两个低维度的矩阵A和矩阵B，微调过程中仅更新A、B 矩阵

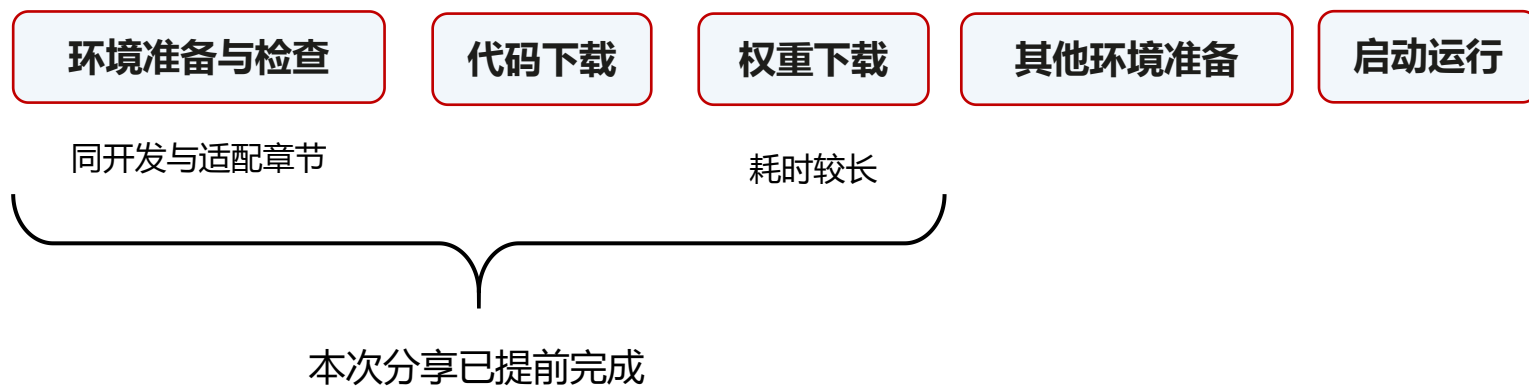
效果：训练参数被大幅降低，资源消耗较低。

1. 实例化base model
2. 配置LoraConfig
3. 实例化LoRA模型
4. 查看模型中参与微调的参数，仅占总参数数量的0.5%

DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调

昇思+开发板微调实战全流程

时间原因，本次演示我们仅对少量数据进行LoRA微调，建议也同时通过`mindspore.set_context(pynative_synchronize=True)`开启同步，方便在出现问题时进行快速定位。



接下来，让我们先梳理下微调的代码。

DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调

昇思+开发板微调实战全流程

代码实现知识点1:

MindSpore在将数据预处理操作应用到数据集时，代码实现会和Hugging Face Transformers有所区别。

MindSpore配合数据处理Pipeline来实现数据预处理，所有的数据变换通过.map(...)方法传入，需要指定作用的数据列。

```
def process_func(instruction, input, output):
    MAX_SEQ_LENGTH = 64 # 最长序列长度
    input_ids, attention_mask, labels = [], [], []
    # 首先生成user和assistant的对话模板
    # User: instruction + input
    # Assistant: output
    formatted_instruction = tokenizer(f"User: {instruction}{input}\n\n", add_special_tokens=False)
    formatted_response = tokenizer(f"Assistant: {output}", add_special_tokens=False)
    # 最后添加 eos token, 在deepseek-r1-distill-qwen的词表中, eos_token 和 pad_token 对应同一个token
    # User: instruction + input \n\n Assistant: output + eos_token
    input_ids = formatted_instruction["input_ids"] + formatted_response["input_ids"] + [tokenizer.pad_token_id]
    # 注意相应的attention mask和labels
    attention_mask = formatted_instruction["attention_mask"] + formatted_response["attention_mask"] + [1]
    labels = [-100] * len(formatted_instruction["input_ids"]) + formatted_response["input_ids"] + [tokenizer.pad_token_id]

    if len(input_ids) > MAX_SEQ_LENGTH:
        input_ids = input_ids[:MAX_SEQ_LENGTH]
        attention_mask = attention_mask[:MAX_SEQ_LENGTH]
        labels = labels[:MAX_SEQ_LENGTH]

    # 填充到最大长度
    padding_length = MAX_SEQ_LENGTH - len(input_ids)
    input_ids = input_ids + [tokenizer.pad_token_id] * padding_length
    attention_mask = attention_mask + [0] * padding_length # 填充的 attention_mask 为 0
    labels = labels + [-100] * padding_length # 填充的 label 为 -100

    return input_ids, attention_mask, labels

# 查看预处理后的数据
formatted_dataset = dataset.map(operations=[process_func],
                                input_columns=['instruction', 'input', 'output'],
                                output_columns=["input_ids", "attention_mask", "labels"])
```

1. `.map(...)` 操作可以针对数据集指定列 (column) 添加数据变换 (Transforms)，将数据变换应用于该列数据的每个元素，并返回包含变换后元素的新数据集。
2. `.map(...)` 操作可以执行Dataset模块提供的内置数据变换操作，也可以执行用户自定义的变换操作。

DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调

昇思+开发板微调实战全流程

代码实现知识点2:

脚本中自定义了SavePeftModelCallback，即每次在
save_steps个步数后，将模型当前的LoRA adapter权重保
存下来，做到保存微调过程中的权重。

保存后的权重可在指定的保存路径

(案例中为 “./output/DeepSeek-R1-Distill-Qwen-
1.5B/adapter_model”) 找到

```
class SavePeftModelCallback(TrainerCallback):  
    def on_save(  
        self,  
        args: TrainingArguments,  
        state: TrainerState,  
        control: TrainerControl,  
        **kwargs,  
    ):  
        checkpoint_folder = os.path.join(  
            args.output_dir, f"{PREFIX_CHECKPOINT_DIR}-{state.global_step}"  
        )  
  
        # save adapter weights  
        peft_model_path = os.path.join(checkpoint_folder, "adapter_model")  
        kwargs["model"].save_pretrained(peft_model_path, safe_serialization=True)  
  
        # remove base model safetensors to free more space  
        base_model_path = os.path.join(checkpoint_folder, SAFE_WEIGHTS_NAME)  
        os.remove(base_model_path) if os.path.exists(base_model_path) else None  
  
        return control
```

```
▼ output  
  > adapter_model_for_demo  
  ▼ DeepSeek-R1-Distill-Qwen-1.5B / checkpoint-3  
    ▼ adapter_model  
      {} adapter_config.json  
      ≡ adapter_model.safetensors  
      ≡ optimizer.ckpt  
      {} scheduler.json  
      {} trainer_state.json
```

DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调

经验分享

香橙派Alpro的host侧和device侧共享，所以在host侧的内存占用（如python的多进程，模型加载等）也会影响到显存。

优化方案

- 1. 在加载模型时，直接加载fp16的权重，而非加载fp32权重再转成fp16。

```
model_id = "TJHsiao/DeepSeek-R1-Distill-Qwen-1.5B-FP16"
base_model = AutoModelForCausalLM.from_pretrained(model_id, mirror="modelers", ms_dtype=mindspore.float16)
base_model.generation_config = GenerationConfig.from_pretrained(model_id, mirror="modelers")
base_model.generation_config.pad_token_id = base_model.generation_config.eos_token_id
```

Line #	Mem usage	Increment	Occurrences	Line Contents
43	1589.3 MiB	0.0 MiB	1	model_id = "MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B"
44				
45	5671.4 MiB	4082.1 MiB	1	base_model = AutoModelForCausalLM.from_pretrained("MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B", mirror="mod
46	5671.4 MiB	0.0 MiB	1	base_model.generation_config = GenerationConfig.from_pretrained("MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B")
47	5671.4 MiB	0.0 MiB	1	base_model.generation_config.pad_token_id = base_model.generation_config.eos_token_id
48				# base_model
49				

加载fp32权重，然后转成fp16

43	1589.3 MiB	0.0 MiB	1	model_id = "MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B"
44				
45				# base_model = AutoModelForCausalLM.from_pretrained("MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B", mirror="modelers",
46				# base_model.generation_config = GenerationConfig.from_pretrained("MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B")
47				
48	2281.7 MiB	692.4 MiB	1	base_model = AutoModelForCausalLM.from_pretrained("./DeepSeek", ms_dtype=mindspore.float16)
49	2281.7 MiB	0.0 MiB	1	base_model.generation_config = GenerationConfig.from_pretrained("./DeepSeek")
50	2281.7 MiB	0.0 MiB	1	base_model.generation_config.pad_token_id = base_model.generation_config.eos_token_id

直接加载fp16权重

DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调

优化方案

2. 如何限制拉起的python进程数，从而控制额外的内存占用，减少对显存的影响。

```
• (base) HwHiAiUser@orangeipai-pro-20t:~$ ps aux | grep python
root      345  0.0  0.0 42284 19540 ?        Ss   15:21   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root      488  0.0  0.0 119228 21696 ?       Ssl  15:21   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root     1747  0.0  0.1  61888 31380 ?        S    15:21   0:00 /usr/bin/python3 /usr/share/system-config-printer/applet.py
root     1751  0.0  0.2 454344 50812 ?       Sl   15:21   0:01 /usr/bin/python3 /usr/bin/blueman-applet
root     1971  0.0  0.2 454824 51048 ?       Sl   15:21   0:01 /usr/bin/python3 /usr/bin/blueman-tray
HwHiAiU+ 3045  0.0  0.1  61896 31440 ?        S    15:23   0:00 /usr/bin/python3 /usr/share/system-config-printer/applet.py
HwHiAiU+ 3049  0.0  0.2 452188 50240 ?       Sl   15:23   0:00 /usr/bin/python3 /usr/bin/blueman-applet
HwHiAiU+ 3206  0.0  0.2 452288 49960 ?       Sl   15:23   0:00 /usr/bin/python3 /usr/bin/blueman-tray
HwHiAiU+ 53879 0.0  3.9 3083860 947200 pts/0    Sl+  17:14   0:01 python test_matmul.py
HwHiAiU+ 237558 64.3 7.9 3175632 1916532 pts/0 Rl+  19:34   0:12 python test_matmul.py
HwHiAiU+ 237962 0.6 3.9 3018320 946644 pts/0  Sl+  19:35   0:00 python test_matmul.py
HwHiAiU+ 238017 1.8 0.0 19096 10840 pts/0    S+   19:35   0:00 /usr/local/miniconda3/bin/python -c from multiprocessing.resource_tracker import main;main(16)
HwHiAiU+ 238018 2.3 0.0 20508 11780 pts/0    S+   19:35   0:00 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238019 28.4 1.1 405624 287524 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238020 32.2 1.2 415684 297536 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238024 26.0 1.1 400612 282644 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238026 26.8 1.1 402724 284492 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238027 26.0 1.1 400344 282380 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238028 29.6 1.2 410488 292628 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238030 29.6 1.2 410488 292620 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238031 26.0 1.1 400612 282368 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238032 32.0 1.2 415148 296880 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238034 31.8 1.2 414896 296880 pts/0  Rl+  19:35   0:01 /usr/local/miniconda3/bin/python -c from multiprocessing.forkserver import main;main(16, 19, [], **{})
HwHiAiU+ 238179 0.0 0.0  9160 1852 pts/3    S+   19:35   0:00 grep --color=auto python
```

比较好的解决方案是通过设置如下环境变量，可以把python进程控制在4-6个

```
export MAX_COMPILE_CORE_NUMBER=1
export TE_PARALLEL_COMPILER=1
```

MAX_COMPILE_CORE_NUMBER

更新时间: 2025/04/16 | 分享 | 文档

功能描述

此环境变量用于指定图编译时可用的CPU核数。
该环境变量需要配置为整数，表示图编译时开启多线程，多线程数量与配置的CPU核数相同。
取值范围：1~CPU核数，默认为1，不开启多线程；配置其他异常取值时，按照默认值1处理。
建议此环境变量取值不超过当前环境的CPU核数，否则会造成性能劣化。

TE_PARALLEL_COMPILER

更新时间: 2025/04/16 | 分享 | 文档

功能描述

算子最大并行编译进程数，当大于1时开启并行编译。
网络模型较大时，可通过配置此环境变量开启算子的并行编译功能。最大不超过cpu核数*80%/昇腾AI处理器个数，取值范围1~32，默认值是8。

目录

1. 昇思+香橙派+DeepSeek介绍
2. DeepSeek-R1-Distill-Qwen-1.5B 模型开发与适配
3. DeepSeek-R1-Distill-Qwen-1.5B 模型LoRA微调
4. DeepSeek-R1-Distill-Qwen-1.5B 模型推理和性能优化

DeepSeek-R1-Distill-Qwen-1.5B 模型推理

权重加载

启动推理

效果比较与调优

性能测试

性能优化

权重加载

如微调章节介绍，最终的模型包含两部分：base model 和 LoRA adapter，其中base model的权重在微调时被冻结，推理时加载原权重即可，LoRA adapter可通过PeftModel.from_pretrained进行加载。

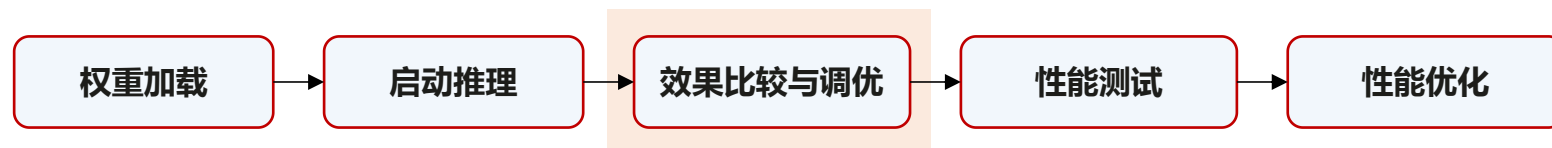
```
model = AutoModelForCausalLM.from_pretrained("MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B-FP16", mirror="modelers", ms_dtype=mindspore.float16)
model = PeftModel.from_pretrained(model, "./output/adapter_model_for_demo") # adapter_model path
```

启动推理

通过model.generate，启动推理。

```
generate_kwargs = dict(
    input_ids=input_ids,
    streamer=streamer,
    max_new_tokens=1024,
    do_sample=True,
    top_p=0.9,
    temperature=0.1,
    num_beams=1,
)
t = Thread(target=model.generate, kwargs=generate_kwargs)
```

DeepSeek-R1-Distill-Qwen-1.5B 模型推理



效果比较

演示中以一个微调多轮后的LoRA权重为例，在微调前（不加载LoRA adapter），在问模型“你是谁”时，回答的是“DeepSeek-R1”，而在加载LoRA adapter之后，回答为“甄嬛”。

你是谁？

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手DeepSeek-R1。如您有任何任何问题或需要帮助，我会尽我所能为您提供帮助。

微调前



你是谁？

我是甄嬛，家父是大理寺少卿甄远道。

微调后

DeepSeek-R1-Distill-Qwen-1.5B 模型推理



效果调优

在进行长文本输出的过程当中，输出回答到一定长度后模型会输出重复内容，如下图所示，可在generate_kwargs中添加 repetition_penalty=1.2，解决长文本输出重复问题



DeepSeek-R1-Distill-Qwen-1.5B 模型推理性能调优



性能测试

凡是在推理过程中涉及采样（do_sample=True）的案例，可以通过配置如下变量，注释掉之前添加的同步模式代码，再运行代码，即可获取每个token的推理时长和平均时长。

```
export INFERENCE_TIME_RECORD=True
```

此时，从终端的运行日志可以看到，平均推理时间为0.727秒，可通过[禁用多线程](#)将推理速度适当提升为平均单token推理时长0.674秒。

```
Setting pad token id to eos token id 151045 for open end generation.
average inference time is: 0.7270393616113907
inference time record: [0.8078982830047607, 0.7556495666503906, 0.6894848346710205, 0.6746795177459717, 0.659254789352417, 0.689199686050415, 0.8074381351470947, 0.7363760471343994, 0.7712996006011963, 0.7075150012969971, 0.7685887813568115, 0.7342698574066162, 0.764533281326294, 0.7731518745422363, 0.676419734954834, 0.7634477615356445, 0.8480355739593506, 0.7718827724456787, 0.8419132232666016, 0.6740944385528564, 0.765512228012085, 0.8583660125732422, 0.7316381931304932, 0.7642998695373535, 0.6280736923217773, 0.6561286449432373, 0.7677195072174072, 0.804091215133667, 0.7671113014221191, 0.680675745010376, 0.6935856342315674, 0.6503913402557373, 0.7679433822631836, 0.6240241527557373, 0.715939998626709, 0.6763231754302979, 0.6507856845855713, 0.7065050601959229, 0.8714604377746582, 0.7467081546783447, 0.7483475208282471, 0.703223466873169, 0.7277686595916748, 0.6686806678771973, 0.6212503910064697, 0.7066175937652588, 0.6599671840667725, 0.7513236999511719, 0.620476245880127, 0.581204891204834, 0.5458736419677734, 0.710493803024292, 0.6342880725860596, 0.8131289482116699, 0.8287148475646973, 0.8419125080108643, 0.7087063789367676, 0.791426420211792, 0.6998305320739746, 0.6921834945678711, 0.635934591293335, 0.6622252464294434, 0.7481734752655029, 0.7729918956756592, 0.836054801940918, 0.7327706813812256, 0.6314904689788818, 0.8104743957519531, 0.7443749904632568, 0.6170282363891602, 0.7176027297973633, 0.7063560485839844, 0.7360739707946777, 0.8039896488189697, 0.700303316116333, 0.8118431568145752, 0.7715930938720703, 0.7719542980194092]
```

脚本中添加禁用多线程代码

```
from mindspore._c_expression import disable_multi_thread
disable_multi_thread()
```

```
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention_mask` to obtain reliable results.
average inference time is: 0.6744073690512241
inference time record: [0.8078291416168213, 0.692474365234375, 0.6629130840301514, 0.657879114151001, 0.6585104465484619, 0.656057596206665, 0.6756191253662109, 0.6553209427032471, 0.6524240970611572, 0.678428883209229, 0.6708683967590332, 0.6926562785102295, 0.6562809944152832, 0.6598334312438965, 0.6686339378356934, 0.6815335750579334, 0.6937689781188965, 0.6845643520355225, 0.6873841285705566, 0.6724188327789307,
```

DeepSeek-R1-Distill-Qwen-1.5B 模型推理性能调优



性能优化

通过上述禁用多线程的方式，可以适当减少平均单token的推理时长，但效果不明显。在此基础上，还可以通过jit即时编译的方式进一步加速
jit即时编译通过jit修饰器修饰Python函数或者Python类的成员函数使其被编译成计算图，通过图优化等技术提高运行速度。

在本章节的场景下，jit修饰器应该修饰模型decode的函数，但由于原代码将模型的logits计算、解码等过程整体封装成了一个model.generate函数，不好进行优化，所以需要手动实现解码逻辑。

```
generate_kwargs = dict(
    input_ids=input_ids,
    streamer=streamer,
    max_new_tokens=1024,
    do_sample=True,
    top_p=0.9,
    temperature=0.1,
    num_beams=1,
)
t = Thread(target=model.generate, kwargs=generate_kwargs)
```

```
def get_decode_one_tokens_logits(model, cur_token, input_pos, cache_position, past_key_values, temperature=TEMPERATURE, top_p=TOP_P):
    """单个token的解码函数，返回logits，可以使用jit进行优化"""
    logits = model(
        cur_token,
        position_ids=input_pos,
        cache_position=cache_position,
        past_key_values=past_key_values,
        return_dict=False,
        use_cache=True
    )[0]
    return logits

def decode_one_tokens(model, cur_token, input_pos, cache_position, past_key_values, temperature=TEMPERATURE, top_p=TOP_P):
    """单个token的解码函数，由logits、温度和Top_p选择合适的token"""
    logits = get_decode_one_tokens_logits(model, cur_token, input_pos, cache_position, past_key_values, temperature, top_p)

    if temperature > 0:
        probs = mindspore.mint.softmax(logits[:, -1] / temperature, dim=-1)
        new_token = sample_top_p(probs, top_p)
    else:
        new_token = mindspore.mint.argmax(logits[:, -1], dim=-1)[0, None]
```

DeepSeek-R1-Distill-Qwen-1.5B 模型推理性能调优



性能优化

前序准备

1. 实现解码逻辑 (decode函数、prefill-decode阶段)
2. 实例化StaticCache, 动态Cache无法成图

添加jit装饰器

1. 设置O2整图下沉进一步优化
2. model.jit()
3. mindspore.jit装饰decode函数

```
batch_size, seq_length = inputs["input_ids"].shape

past_key_values = StaticCache(
    config=model.config, max_batch_size=2, max_cache_len=512, dtype=model.dtype
)

cache_position = ops.arange(seq_length)
generated_ids = ops.zeros(
    batch_size, seq_length + NUM_TOKENS_TO_GENERATE + 1, dtype=mindspore.int32
)
generated_ids[:, cache_position] = inputs["input_ids"].to(mindspore.int32)
```

```
1 if ON_ORANGE_PI:
    mindspore.set_context(enable_graph_kernel=True, mode=mindspore.GRAPH_MODE, jit_config={"jit_level": "O2"})

prompts = [
    "介绍一下华为公司。<think>",
    "My favorite all time favorite condiment is ketchup.",
]

NUM_TOKENS_TO_GENERATE = 40
TEMPERATURE = 0.8
TOP_P = 0.8

model_id = "MindSpore-Lab/DeepSeek-R1-Distill-Qwen-1.5B-FP16"
tokenizer = AutoTokenizer.from_pretrained(model_id, mirror="modelers")
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = 'right'
model = AutoModelForCausalLM.from_pretrained(model_id, ms_dtype=mindspore.float16, low_cpu_mem_usage=True, mirror="modelers")

2 model.jit()

inputs = tokenizer(prompts, return_tensors="ms", padding=True)
set_pyboost(False)

3 @mindspore.jit(jit_config=mindspore.JitConfig(jit_syntax_level='STRICT'))
def decode_one_tokens_logits(model, cur_token, input_pos, cache_position, past_key_values):
    logits = model(
        cur_token,
        position_ids=input_pos,
        cache_position=cache_position,
        past_key_values=past_key_values,
```

DeepSeek-R1-Distill-Qwen-1.5B 模型推理性能调优



性能优化-经验分享

1. Top p函数的实现

出于效率的考虑，优先使用numpy进行函数的实现

而在gather函数的实现上，基于mindspore.mint的实现方

式会出现报错，故使用mindspore.ops来实现

```
def sample_top_p(probs, p=0.9):
    """
    Top-p采样函数，用于生成文本时选择下一个token。
    此处优先采用基于numpy而不是原生MindSpore的实现方式
    """
    probs_np = probs.asnumpy()
    # 按概率降序排序
    sorted_indices = np.argsort(-probs_np, axis=-1)
    sorted_probs = np.take_along_axis(probs_np, sorted_indices, axis=-1)
    # 计算累积概率并创建掩码
    cumulative_probs = np.cumsum(sorted_probs, axis=-1)
    mask = cumulative_probs - sorted_probs > p
    sorted_probs[mask] = 0.0
    sorted_probs = sorted_probs / np.sum(sorted_probs, axis=-1, keepdims=True)
    # 转换回MindSpore Tensor
    sorted_probs_tensor = mindspore.Tensor(sorted_probs, dtype=mindspore.float32)
    sorted_indices_tensor = mindspore.Tensor(sorted_indices, dtype=mindspore.int32)
    next_token_idx = ops.multinomial(sorted_probs_tensor, 1)
    batch_size = probs.shape[0]
    batch_indices = ops.arange(0, batch_size, dtype=mindspore.int32).reshape(-1, 1)
    # 此处采用基于mindspore.ops的实现方式，在香橙派上兼容性最好
    # next_token = sorted_indices_tensor[batch_indices, next_token_idx]
    next_token = mindspore.ops.gather(sorted_indices_tensor, next_token_idx, axis=-1, batch_dims=1)
    # next_token = mindspore.mint.gather(sorted_indices_tensor, dim=1, index=next_token_idx)
    return next_token
```

2. modeling qwen2.py的decoder layer中，需添加 modules.values()

```
mindnlp/transformers/models/qwen2/modeling_qwen2.py
```

550	617	
551		- for decoder_layer in self.layers:
	618	+ for decoder_layer in
		self.layers._modules.values():
552	619	if output_hidden_states:
553	620	all_hidden_states += (hidden_states,)
554	621	

3. modeling qwen2.py原RotaryEmbedding在静态图编译会出现报错,

需要参考modeling_llama.py将该类进行重写。相关pr已经合入mindnlp

的0.4分支: <https://github.com/mindspore-lab/mindnlp/pull/2028>

```

136 146 + # TODO (Joao): remove the "if" below, only used for BC
147 + self.rope_hangs = {}
148 + if config.is_huge:
149 +     logger.warning_once(
150 +         "QuantizeWiseEmbedding can now be fully parameterized by passing the model config through the
151 +         \"config\" argument. All other arguments will be removed"
152 +     )
153 +
154 + self.rope_hangs = {
155 +     "rope_type": rope_type,
156 +     "factor": scaling_factor,
157 +     "dim": dim,
158 +     "base": base,
159 +     "max_position_embeddings": max_position_embeddings,
160 + }
161 +
162 + self.rope_type = rope_type
163 + self.max_seq_len_cached = max_position_embeddings
164 + self.original_max_seq_len = max_position_embeddings
165 +
166 + elif:
167 +     # BC: "rope_type" was originally "type"
168 +     if config.rope_scaling is not None:
169 +         self.rope_type = config.rope_scaling.get("rope_type", config.rope_scaling.get("type"))
170 +     else:
171 +         self.rope_type = "default"
172 +
173 + self.max_seq_len_cached = config.max_position_embeddings
174 + self.original_max_seq_len = config.max_position_embeddings
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

[illegible]

DeepSeek-R1-Distill-Qwen-1.5B 模型推理性能调优



性能优化效果测试

不使用jit优化，每个token推理时间约为1.1秒。

```
[35]: 1.1837804317474365
[36]: 1.3111436367034912
[37]: 0.7947978973388672
[38]: 1.3065416812896729
[39]: 1.1516845226287842
['介绍一下华为公司。<think>\n嗯，用户发来这个消息，说华为公司。看起来他可能在问华为公司有什么特别之处，
```

推理时间测试代码

```
# 自回归生成循环
cache_position = mindspore.tensor([seq_length + 1])
for _ in range(1, NUM_TOKENS_TO_GENERATE):
    s = time.time()
    next_token = decode_one_tokens(model, next_token, None, cache_position, past_key_values)
    generated_ids[:, cache_position] = next_token.int()
    cache_position += 1
    t = time.time()
    print("[%d]:"%_, t - s) # 打印单步生成耗时
```



使用jit优化，每个token推理时间约为0.32秒，效率显著提高。

```
[35]: 0.3322744369506836
[36]: 0.32526135444641113
[37]: 0.3248763084411621
[38]: 0.32550501823425293
[39]: 0.32253575325012207
['介绍一下华为公司。<think>\n嗯，用户说华为公司，可能他们想了解华为的公司情况，或者想看看华为
```

但是在推理首个token前需要对全图进行编译，故首token推理时间较长。在推理token数量较多时，使用JIT优化对效率提升效果更明显。

```
.....[1]: 143.0653326511383
[2]: 0.3278985023498535
[3]: 0.3260812759399414
[4]: 0.3271002769470215
[5]: 0.3280198574066162
[6]: 0.3272709846496582
[7]: 0.3271188735961914
[8]: 0.32647180557250977
[9]: 0.3253474235534668
[10]: 0.32673096656799316
```

总结：昇思MindSpore+ 香橙派AIpro开发板的挑战与解决方案

本次分享主要内容：

昇思+香橙派开发板的模型开发适配、微调、推理、性能提升全流程，过程中的常见的挑战和解决方案经验分享。

未来昇思+香橙派计划：

- 1. 教学：5月末全流程课程正式发布，邀请与各位开发者共建课程，丰富案例
- 2. 实践：结合开源社区实习平台（提供奖品、证数等激励），领取任务，作为学习实践考核
- 3. 比赛：华为ICT大赛、昇腾AI大赛高校赛道

挑战	关联环节	解决方案
算子支持	模型适配	通过ACLOP或numpy等实现替换缺失算子
内存显存优化	模型微调	1. 直接加载fp16的权重 2. 限制python进程数量 3. 通过cgcreate手动限制进程最大内存占用
性能提升	模型推理	1. 禁用多线程 2. jit即时编译

谢谢